VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS
(23CS4PCADA)

Submitted by

AMRUTA SHARANABASAPPA NIMBAL
(1BM24CS037)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by **Amruta Sharanabasappa Nimbal (1BM24CS037)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

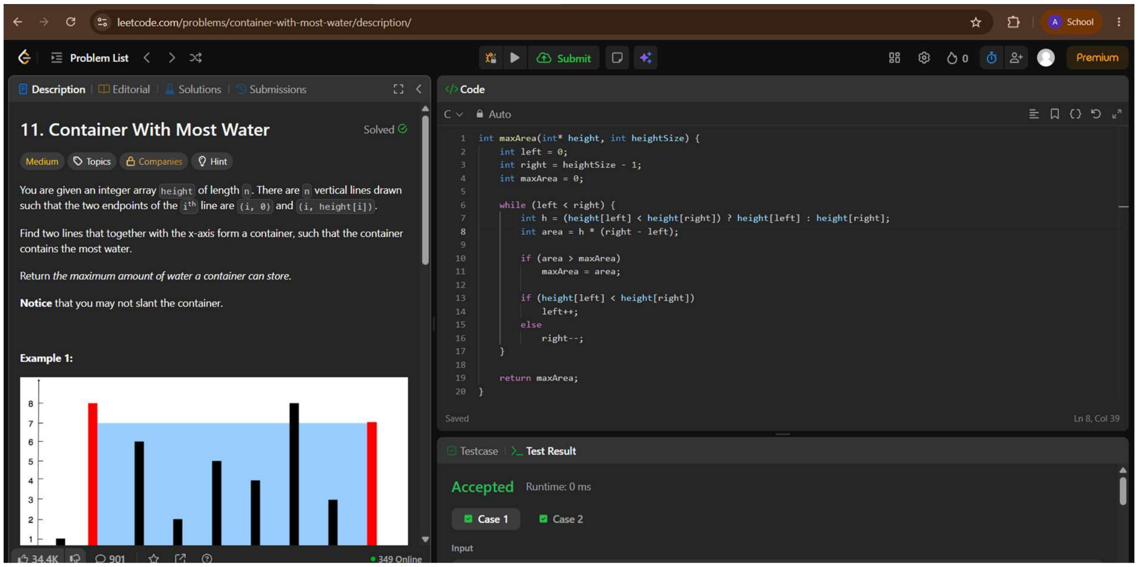
Kanchana M Dixit	**Dr. Kavitha Sooda**
Assistant Professor	Professor and Head
Department of CSE	Department of CSE
BMSCE, Bengaluru	BMSCE, Bengaluru

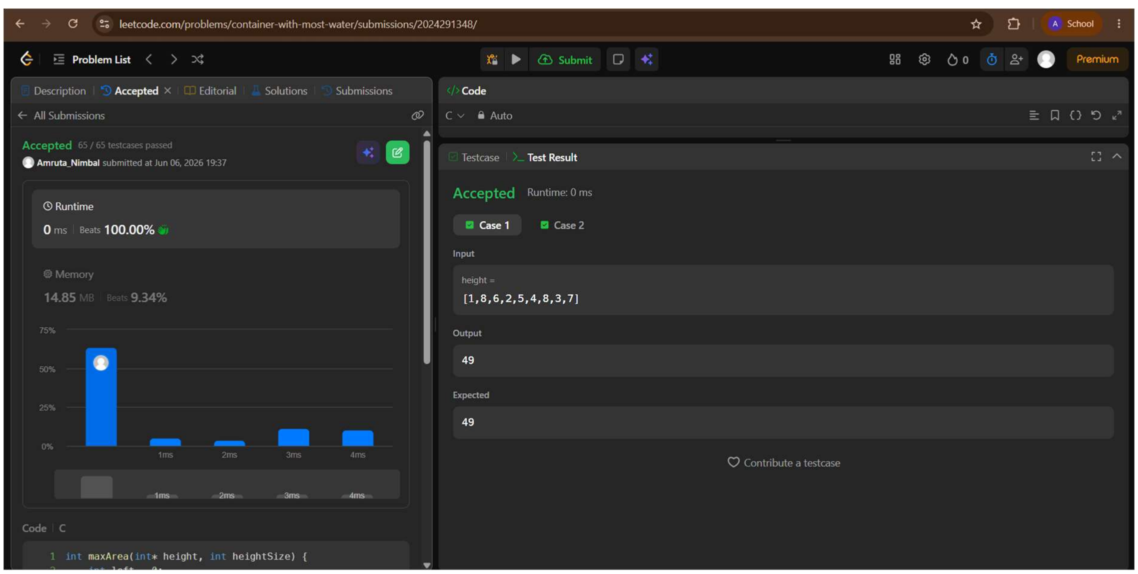
Index Sheet

Course outcomes:

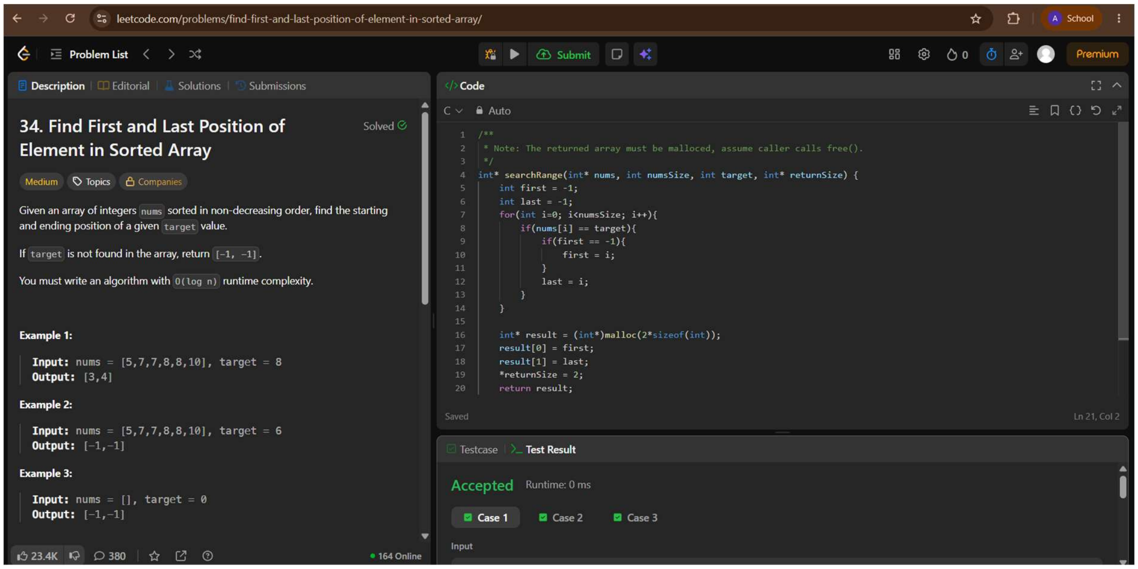
CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

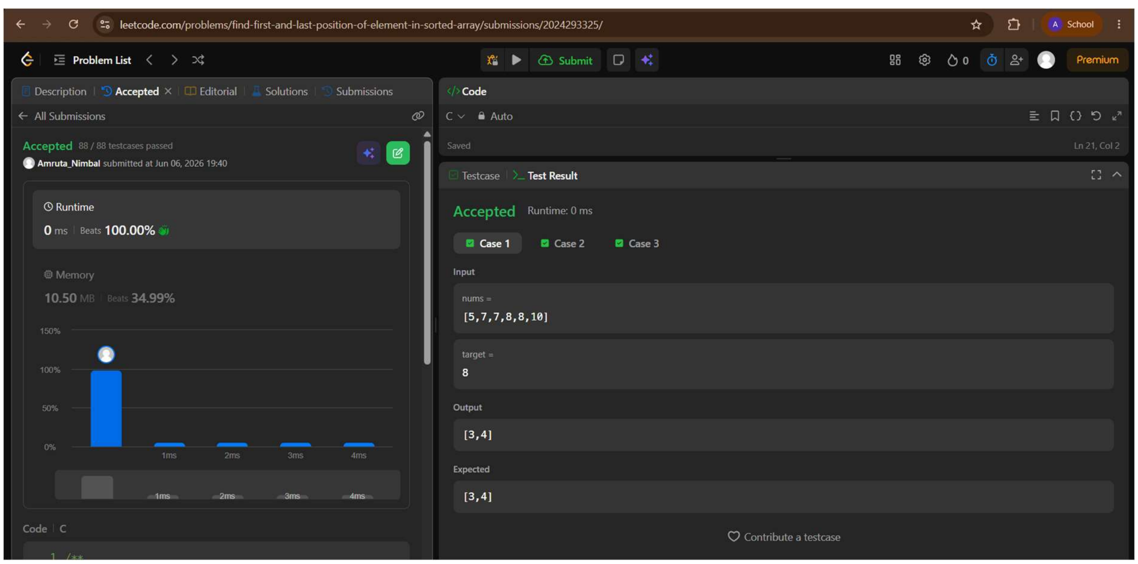
Leetcode 11:


leetcode.com/problems/container-with-most-water/description/
11. Container With Most Water
Solved
Medium
Topics
Companies
Hint
You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).
Find two lines that together with the x-axis form a container, such that the container contains the most water.
Return the maximum amount of water a container can store.
Notice that you may not slant the container.
Example 1:
Code
Accepted
Runtime: 0 ms
Case 1
Case 2
int maxArea(int* height, int heightSize) {
int left = 0;
int right = heightSize - 1;
int maxArea = 0;
while (left < right) {
int h = (height[left] < height[right]) ? height[left] : height[right];
int area = h * (right - left);
if (area > maxArea)
maxArea = area;
if (height[left] < height[right])
left++;
else
right--;
}
return maxArea;
}


leetcode.com/problems/container-with-most-water/submissions/2024291348/
Accepted
65 / 65 testcases passed
Amruta_Nimbal submitted at Jun 06, 2026 19:37
Runtime
0 ms
Beats 100.00%
Memory
14.85 MB
Beats 9.34%
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
height =
[1,8,6,2,5,4,8,3,7]
Output
49
Expected
49
Contribute a testcase

Leetcode 34:


leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/
Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
34. Find First and Last Position of Element in Sorted Array
Solved
Medium
Topics
Companies
Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.
If target is not found in the array, return [-1, -1].
You must write an algorithm with O(log n) runtime complexity.
Example 1:
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
Example 2:
Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
Example 3:
Input: nums = [], target = 0
Output: [-1,-1]
23.4K
380
164 Online
C
Auto
/**
* Note: The returned array must be malloced, assume caller calls free().
*/
int* searchRange(int* nums, int numsSize, int target, int* returnSize) {
int first = -1;
int last = -1;
for(int i=0; i<numsSize; i++){
if(nums[i] == target){
if(first == -1){
first = i;
}
last = i;
}
}
int* result = (int*)malloc(2*sizeof(int));
result[0] = first;
result[1] = last;
*returnSize = 2;
return result;
Saved
Ln 21, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input


leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/submissions/2024293325/
Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted
88 / 88 testcases passed
Amruta_Nimbal submitted at Jun 06, 2026 19:40
Runtime
0 ms
Beats 100.00%
Memory
10.50 MB
Beats 34.99%
150%
100%
50%
0%
1ms
2ms
3ms
4ms
Code
C
Code
C
Auto
Saved
Ln 21, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[5,7,7,8,8,10]
target =
8
Output
[3,4]
Expected
[3,4]
Contribute a testcase

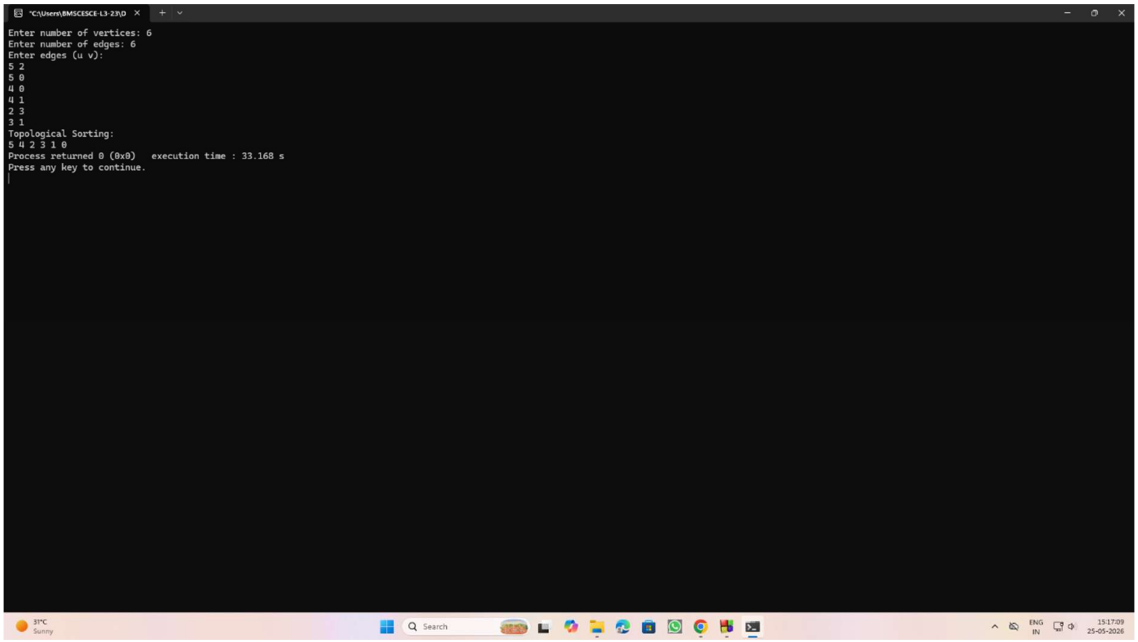
Lab Program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
int graph[10][10], visited[10], stack[10];
int n, top = -1;

void dfs(int v)
{
    visited[v] = 1;
    for(int i = 0; i < n; i++)
    {
        if(graph[v][i] == 1 && !visited[i])
        {
            dfs(i);
        }
    }
    stack[++top] = v;
}

int main()
{
    int edges, u, v;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            graph[i][j] = 0;
```

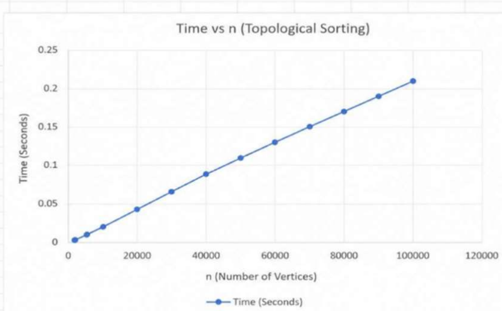
```
        }
    }
    printf("Enter number of edges: ");
    scanf("%d", &edges);
    printf("Enter edges (u v):\n");
    for(int i = 0; i < edges; i++)
    {
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }
    for(int i = 0; i < n; i++)
    {
        if(!visited[i])
        {
            dfs(i);
        }
    }
    printf("Topological Sorting:\n");
    while(top != -1)
    {
        printf("%d ", stack[top--]);
    }
    return 0;
}
```

Output:

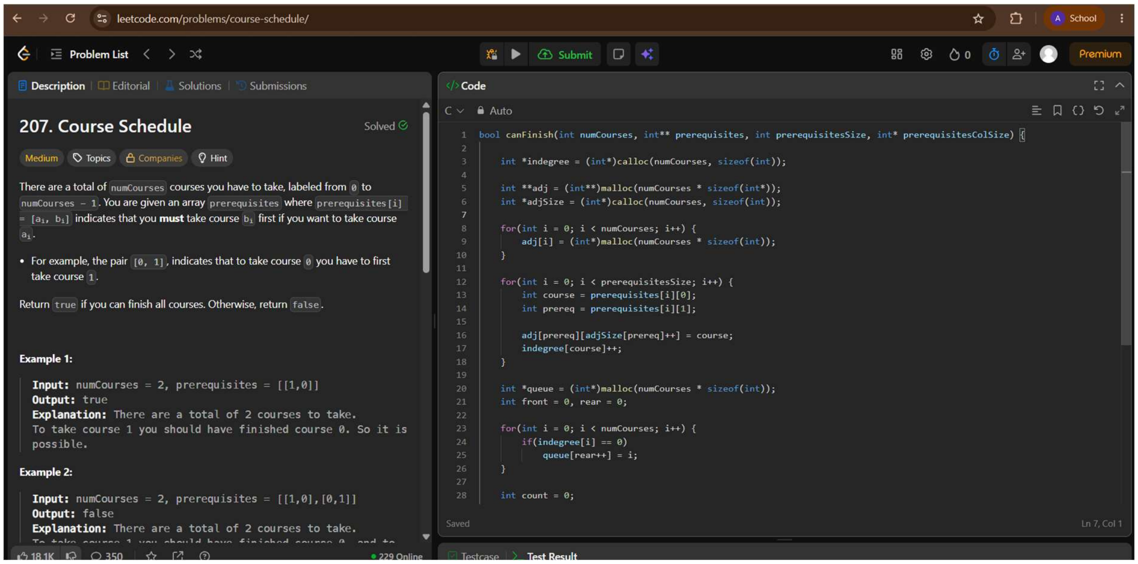


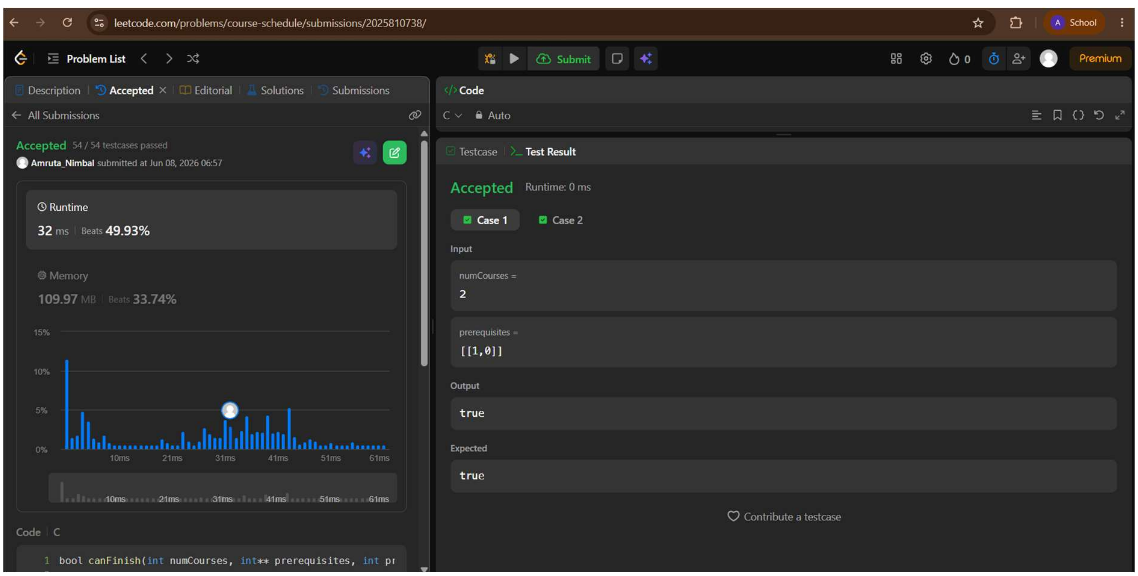
Time Complexity: O(V+E)

n (Number of Vertices)	Time (Seconds)
1000	0.0021
5000	0.0105
10000	0.0208
20000	0.0415
30000	0.0617
40000	0.0826
50000	0.1039
60000	0.1251
70000	0.1463
80000	0.1679
90000	0.1888
100000	0.2102

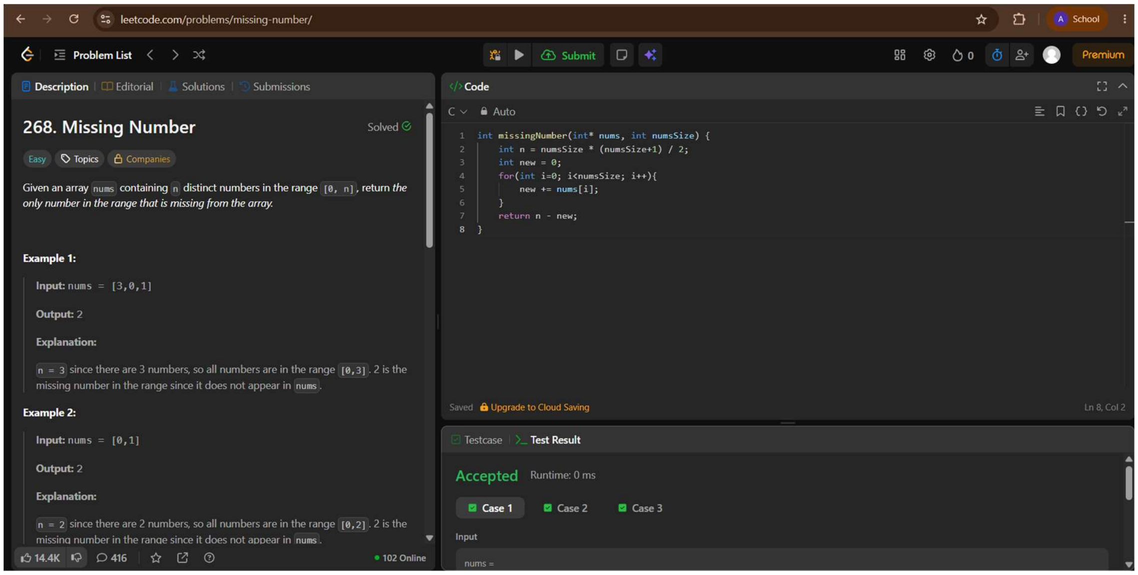


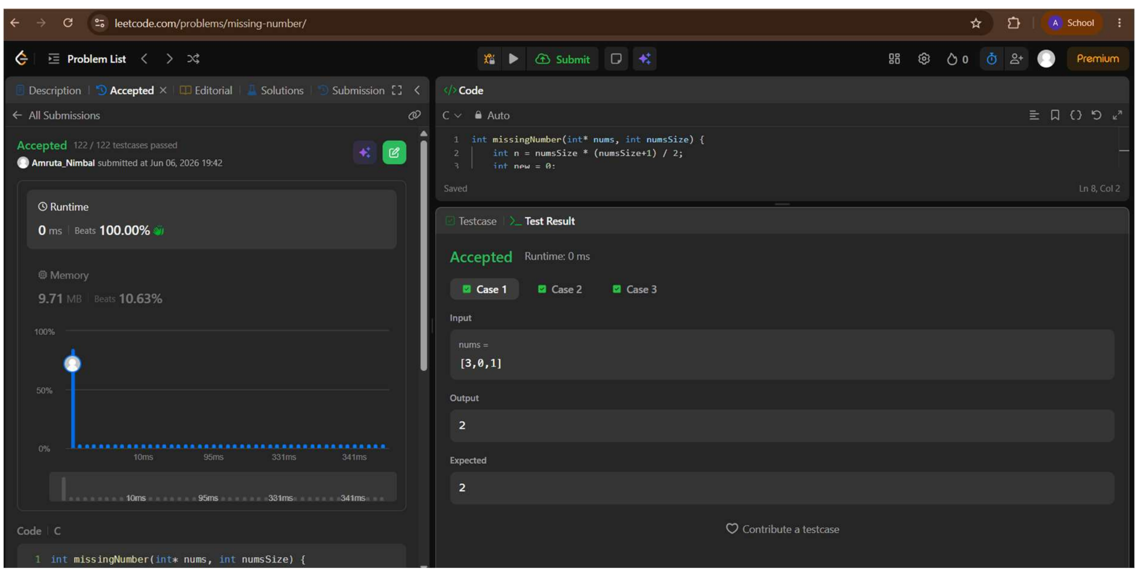
Leetcode 207:



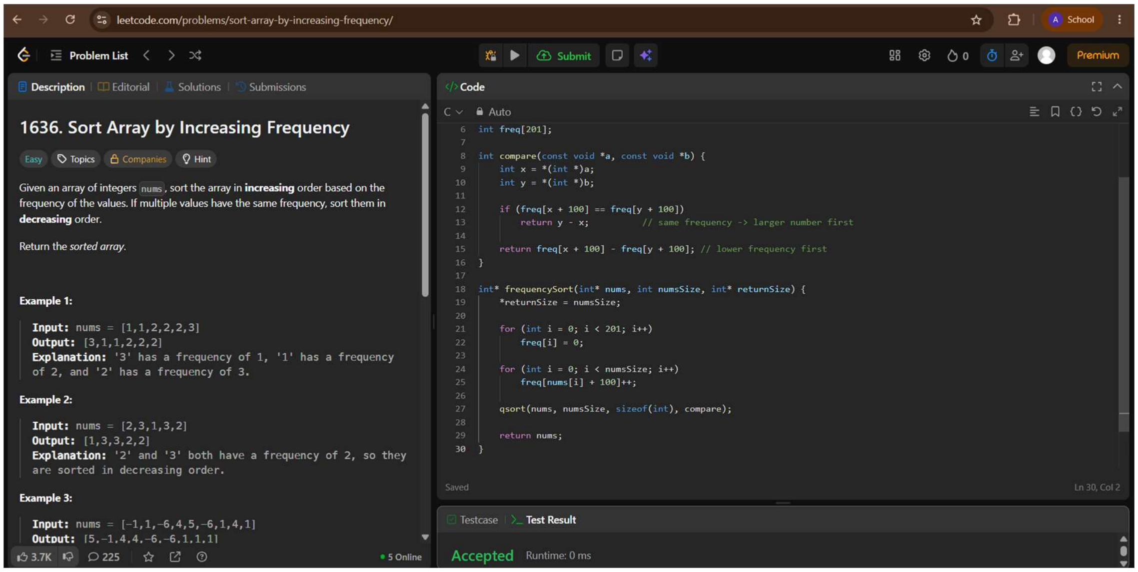


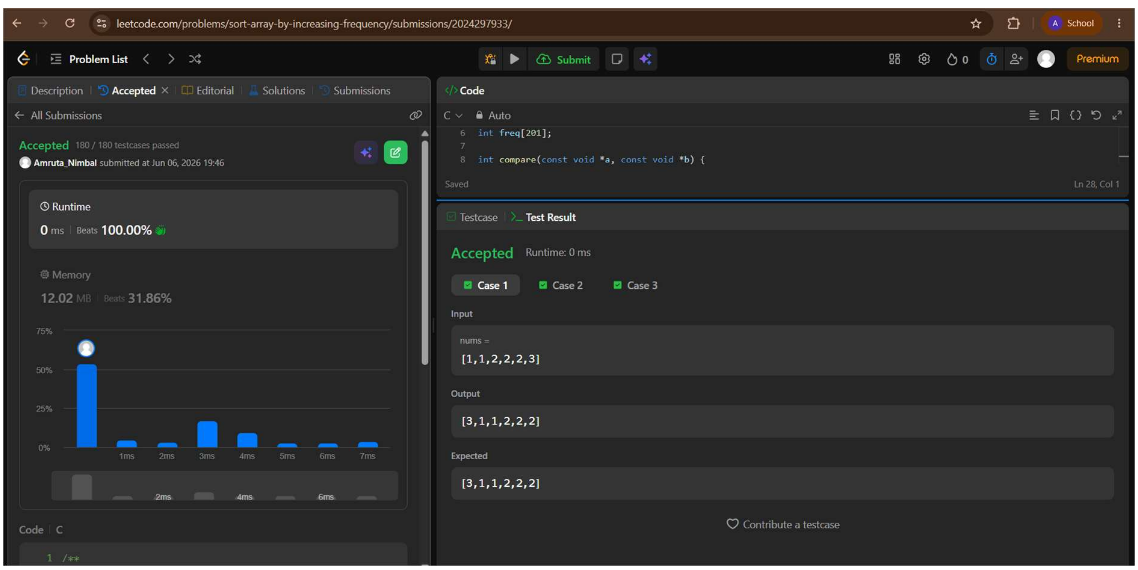
Leetcode 268:





Leetcode 1636:





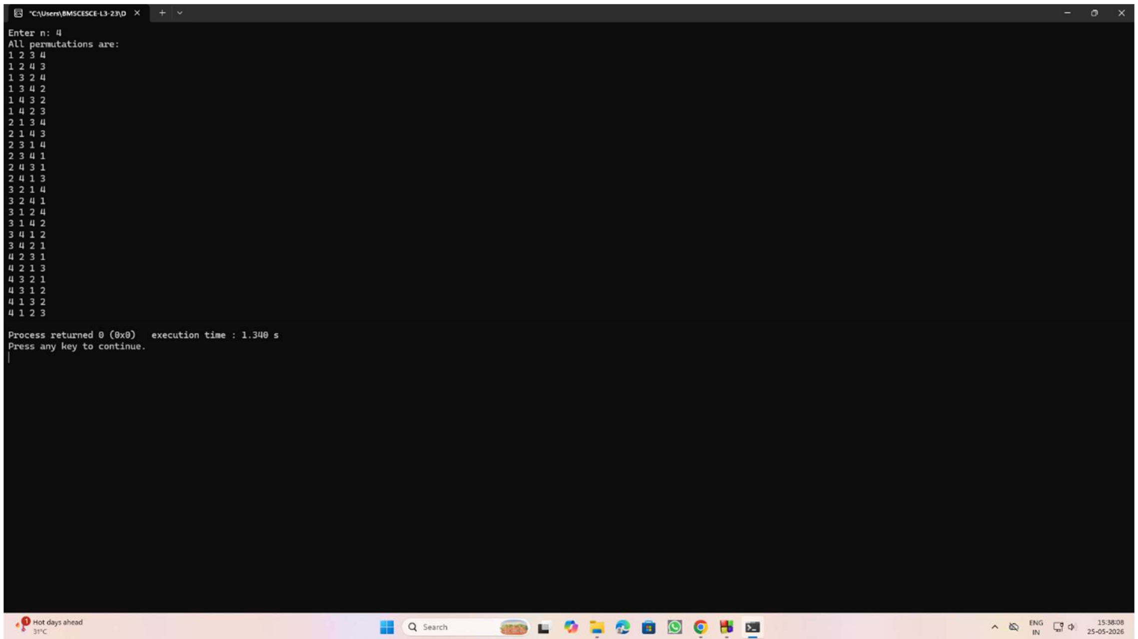
Lab Program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}

void permute(int a[], int start, int end)
{
    int i;
    if(start == end)
    {
        for(i = 0; i <= end; i++)
        {
            printf("%d ", a[i]);
        }
        printf("\n");
    }
    else
    {
        for(i = start; i <= end; i++)
        {
            swap(&a[start], &a[i]);
            permute(a, start + 1, end);
```

```
            swap(&a[start], &a[i]);
        }
    }
}

int main()
{
    int n, i;
    int a[10];
    printf("Enter n: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        a[i] = i + 1;
    }
    printf("All permutations are:\n");
    permute(a, 0, n - 1);
    return 0;
}
```

Output:


"C:\Users\BMSCESCE-L3-23\D
Enter n: 4
All permutations are:
1 2 3 4
1 2 4 3
1 3 2 4
1 3 4 2
1 4 3 2
1 4 2 3
2 1 3 4
2 1 4 3
2 3 1 4
2 3 4 1
2 4 3 1
2 4 1 3
3 2 1 4
3 2 4 1
3 1 2 4
3 1 4 2
3 4 1 2
3 4 2 1
4 2 3 1
4 2 1 3
4 3 2 1
4 3 1 2
4 1 3 2
4 1 2 3
Process returned 0 (0x0)   execution time : 1.340 s
Press any key to continue.
Hot days ahead
31°C
Search
ENG
IN
15:38:08
25-05-2026

Lab Program 3:

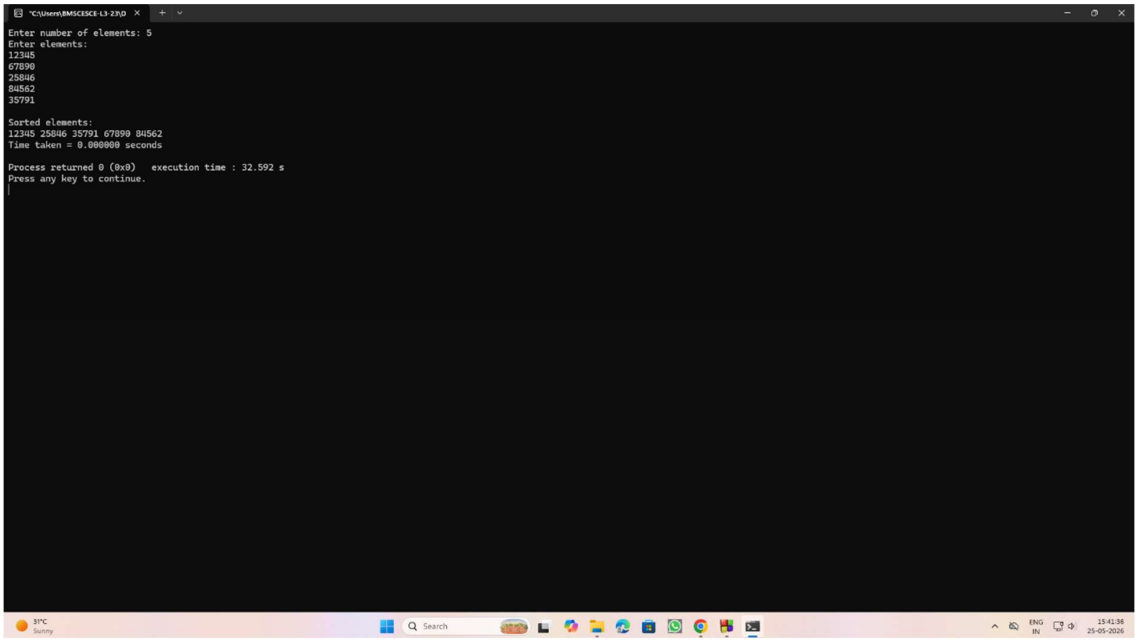
Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>

#include<stdlib.h>

#include<time.h>


void merge(int a[], int f, int m, int l)

{

    int i, j, k;

    i = f;

    j = m + 1;

    k = 0;

    int r[100];

    while(i <= m && j <= l)

    {

        if(a[i] < a[j])

        {

            r[k] = a[i];

            i++;

        }

        else

        {

            r[k] = a[j];

            j++;

        }

        k++;

    }

    while(i <= m)

    {
```

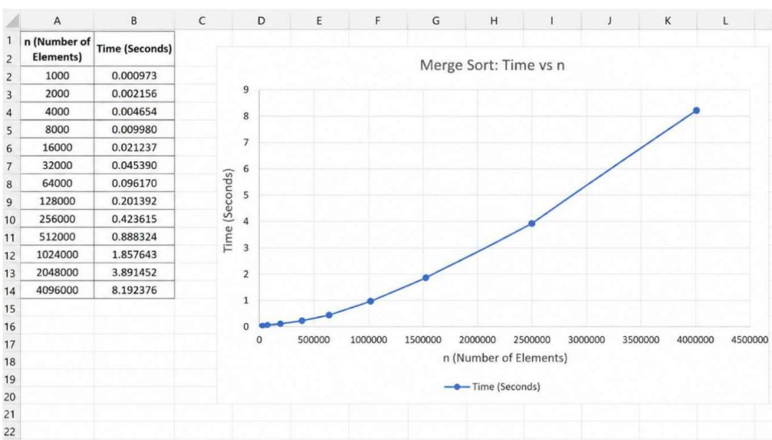
```
        r[k] = a[i];
        i++;
        k++;
    }
    while(j <= l)
    {
        r[k] = a[j];
        j++;
        k++;
    }
    k = 0;
    for(i = f; i <= l; i++)
    {
        a[i] = r[k];
        k++;
    }
}

void mergesort(int a[], int f, int l)
{
    if(f < l)
    {
        int m = (f + l) / 2;
        mergesort(a, f, m);
        mergesort(a, m + 1, l);
        merge(a, f, m, l);
    }
}
```

```
int main()
{
    int n, i;
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    start = clock();
    mergesort(a, 0, n - 1);
    end = clock();
    cpu_time =
    ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted elements:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
    printf("Time taken = %f seconds\n", cpu_time);
    return 0;
}
```

Output:



Time Complexity: O(nlogn)



n (Number of Elements)	Time (Seconds)
1000	0.000973
2000	0.002156
4000	0.004654
8000	0.009980
16000	0.021237
32000	0.045390
64000	0.096170
128000	0.201392
256000	0.423615
512000	0.888324
1024000	1.857643
2048000	3.891452
4096000	8.192376

Lab Program 4:

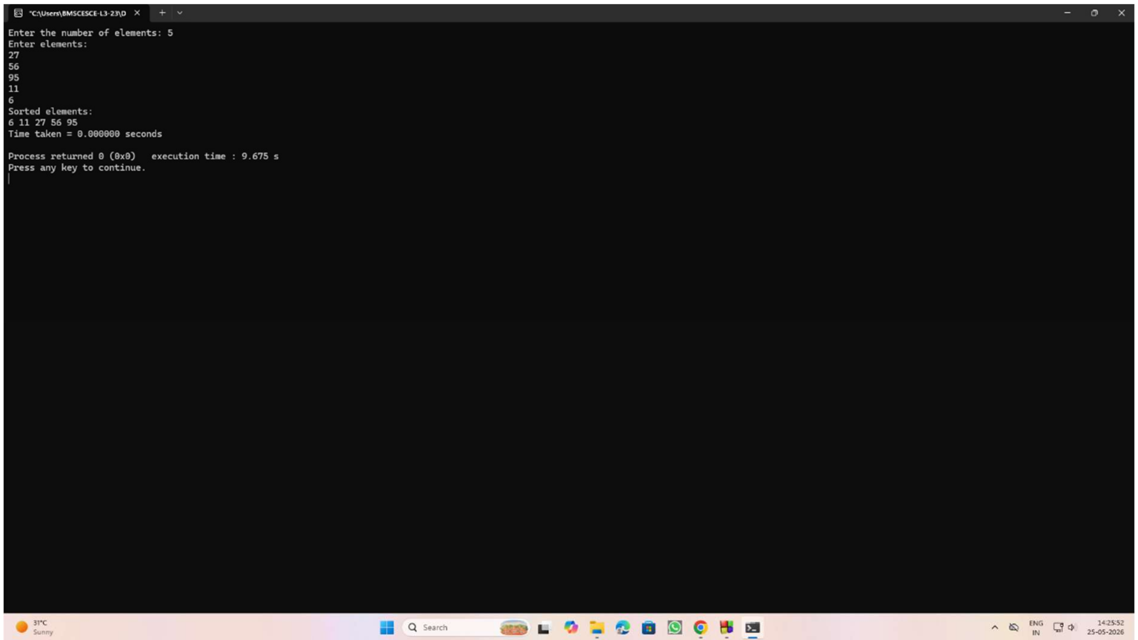
Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include<stdio.h>

#include<stdlib.h>

#include<time.h>


void swap(int *a, int *b)

{

    int temp = *a;

    *a = *b;

    *b = temp;

}


int partition(int a[], int low, int high)

{

    int pivot = a[low];

    int i = low+1;

    int j = high;

    while(1)

    {

        while(i<=high && a[i]<=pivot)

            i++;

        while(a[j]>pivot)

            j--;

        if(i<j)

            swap(&a[i], &a[j]);

        else

            break;

    }
```

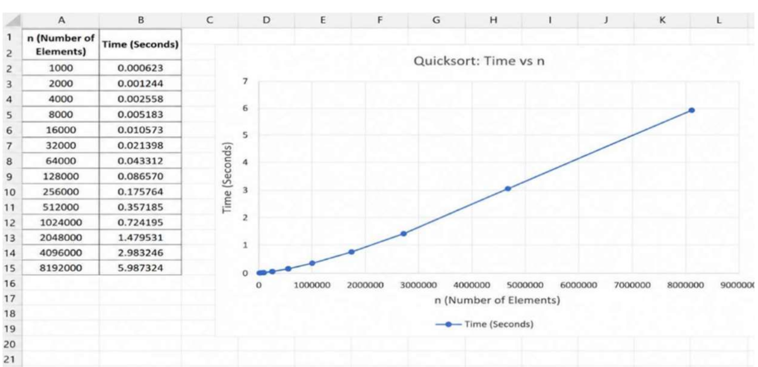
```
    swap(&a[low], &a[j]);
    return j;
}

void quicksort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quicksort(a, low, p - 1);
        quicksort(a, p + 1, high);
    }
}

int main()
{
    int n, i;
    clock_t start, end;
    double cpu_time;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    quicksort(a, 0, n - 1);
    end = clock();
    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted elements:\n");
    for(i = 0; i < n; i++)
```

```
        printf("%d ", a[i]);
    printf("\nTime taken = %f seconds\n", cpu_time);
    return 0;
}
```

Output:



Time Complexity: O(nlogn)



n (Number of Elements)	Time (Seconds)
1000	0.000623
2000	0.001244
4000	0.002558
8000	0.005183
16000	0.010573
32000	0.021398
64000	0.043312
128000	0.086570
256000	0.175764
512000	0.357185
1024000	0.724195
2048000	1.479531
4096000	2.983246
8192000	5.987324

Lab Program 5:

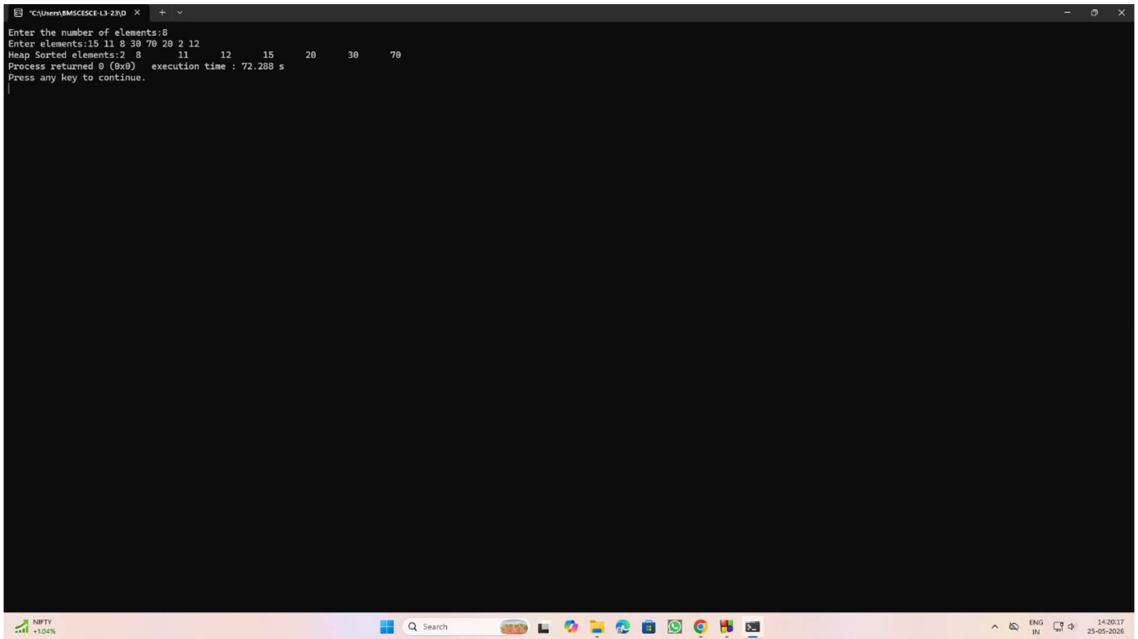
Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>


void heapify(int a[], int n, int p)
{
   int c, item;
   item = a[p];
   c = 2*p+1;
   while(c<=(n-1))
   {
      if((c+1<n) && (a[c]<a[c+1]))
         c++;
      if(item<a[c])
      {
         a[p] = a[c];
         p = c;
         c = 2*p+1;
      }
      else
         break;
   }
   a[p] = item;
}


void build_heap(int a[], int n)
{
   int i;
   for(i=(n-1)/2; i>=0; i--)
```

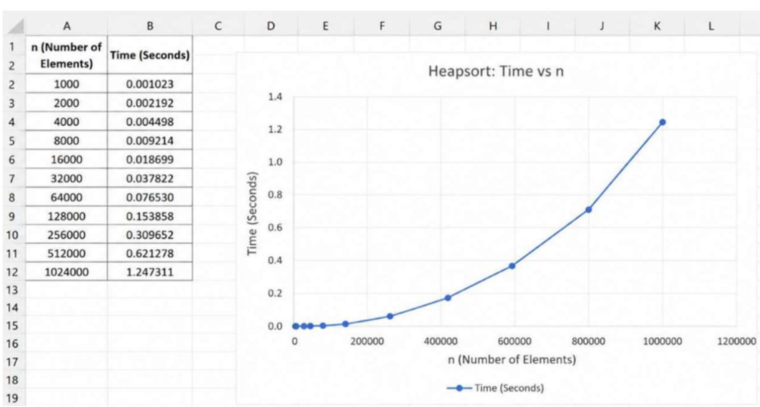
```
        heapify(a, n, i);
}

void heap_sort(int a[], int n)
{
    int i, temp;
    build_heap(a, n);
    for(i=(n-1); i>0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify(a, i, 0);
    }
}

int main()
{
    int n, a[20], i;
    printf("Enter the number of elements:");
    scanf("%d", &n);
    printf("Enter elements:");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    heap_sort(a, n);
    printf("Heap Sorted elements:");
    for(i=0; i<n; i++)
        printf("%d\t", a[i]);
    return 0;
```

}

Output:



Time Complexity: O(nlogn)



n (Number of Elements)	Time (Seconds)
1000	0.001023
2000	0.002192
4000	0.004498
8000	0.009214
16000	0.018699
32000	0.037822
64000	0.076530
128000	0.153858
256000	0.309652
512000	0.621278
1024000	1.247311

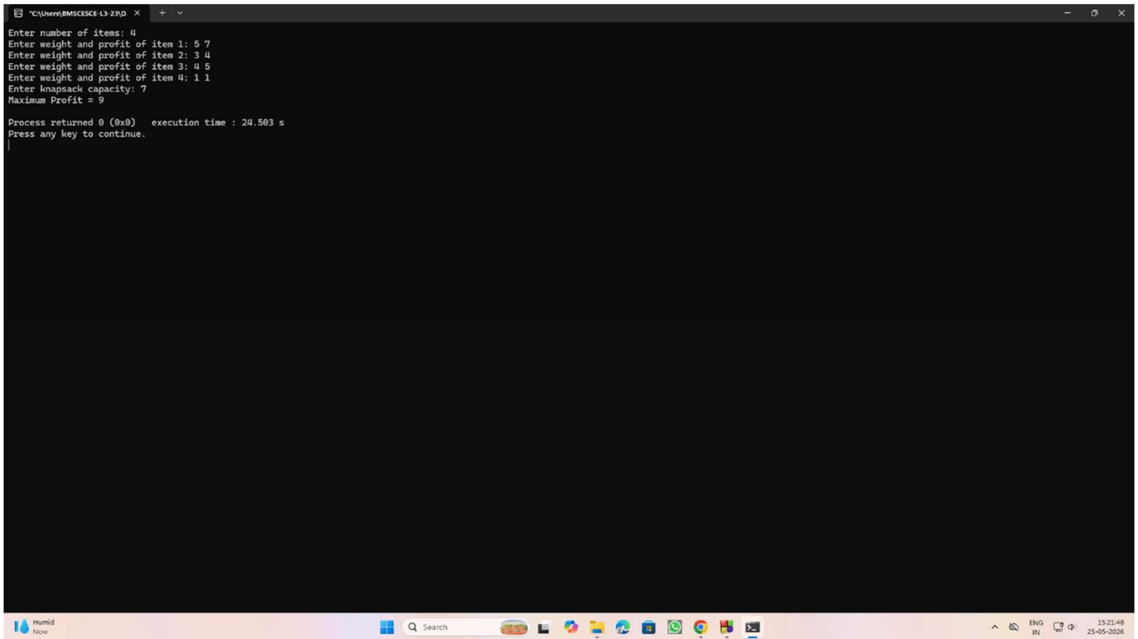
Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

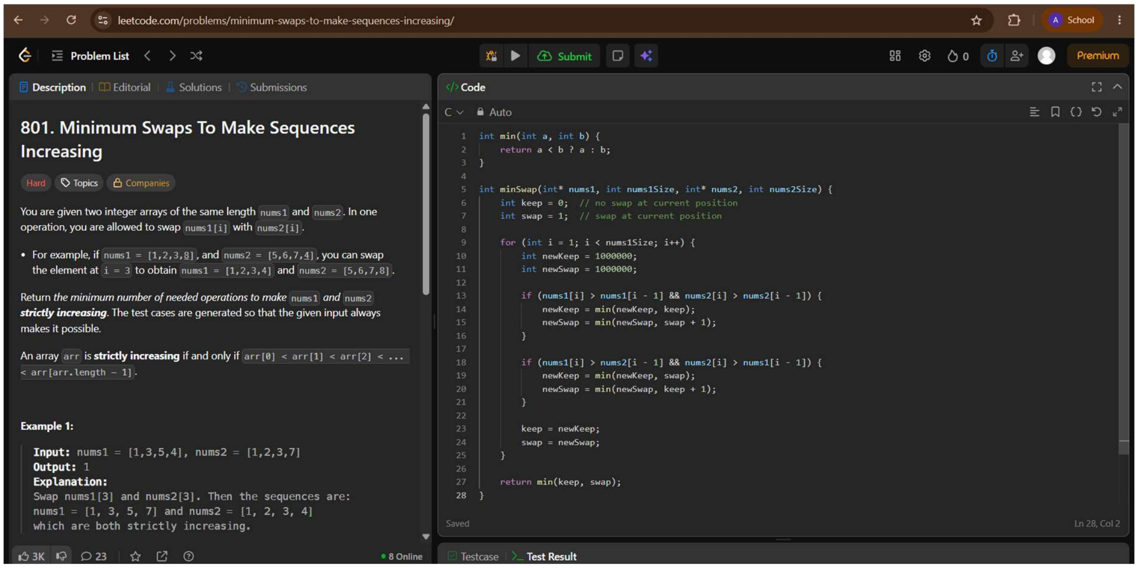
```
#include <stdio.h>

struct Item
{
    int weight;
    int profit;
};

int max(int a, int b)
{
    if(a > b)
        return a;
    else
        return b;
}

int main()
{
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item item[n];
    for(int i = 0; i < n; i++)
    {
        printf("Enter weight and profit of item %d: ", i + 1);
        scanf("%d %d", &item[i].weight, &item[i].profit);
    }
```

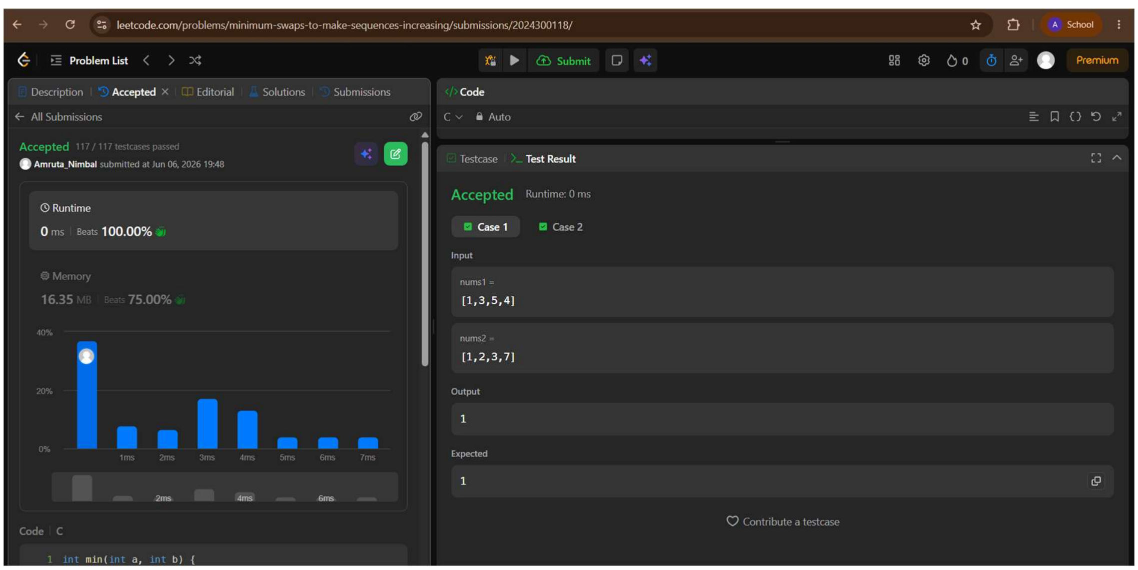
```
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);
    int dp[n + 1][capacity + 1];
    for(int i = 0; i <= n; i++)
    {
        for(int w = 0; w <= capacity; w++)
        {
            if(i == 0 || w == 0)
            {
                dp[i][w] = 0;
            }
            else if(item[i - 1].weight <= w)
            {
                dp[i][w] = max(
                    item[i - 1].profit +
                    dp[i - 1][w - item[i - 1].weight],
                    dp[i - 1][w]
                );
            }
            else
            {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    printf("Maximum Profit = %d\n", dp[n][capacity]);
    return 0;
}
```

Output:


"C:\Users\BMSCESCE-L3-23\D
Enter number of items: 4
Enter weight and profit of item 1: 5 7
Enter weight and profit of item 2: 3 4
Enter weight and profit of item 3: 4 5
Enter weight and profit of item 4: 1 1
Enter knapsack capacity: 7
Maximum Profit = 9
Process returned 0 (0x0)   execution time : 24.503 s
Press any key to continue.

Leetcode 801:





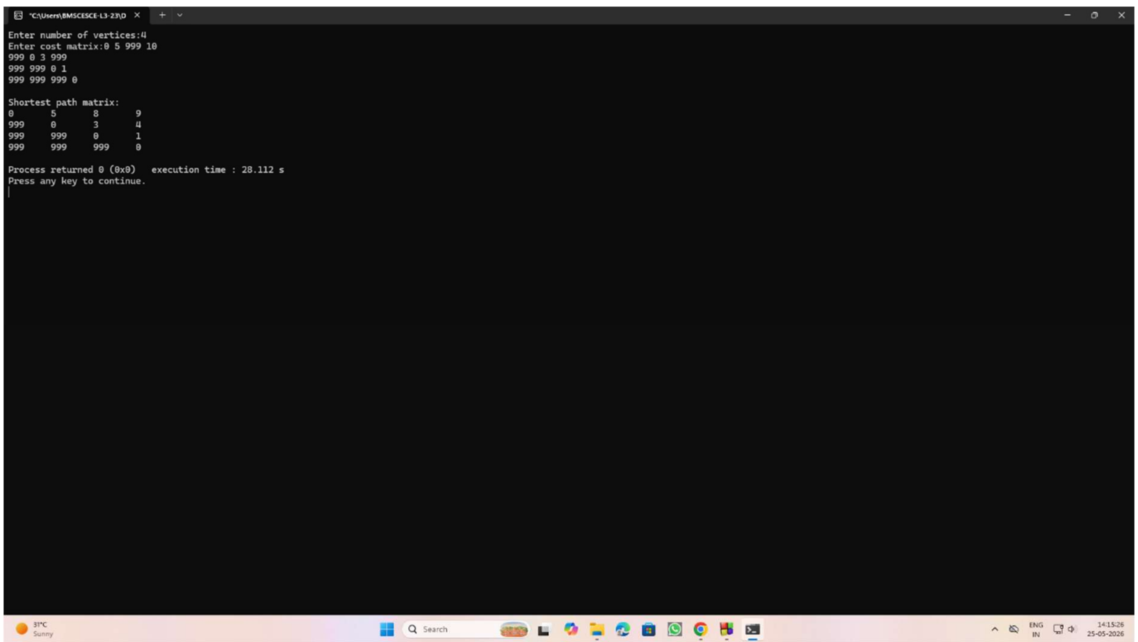
Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

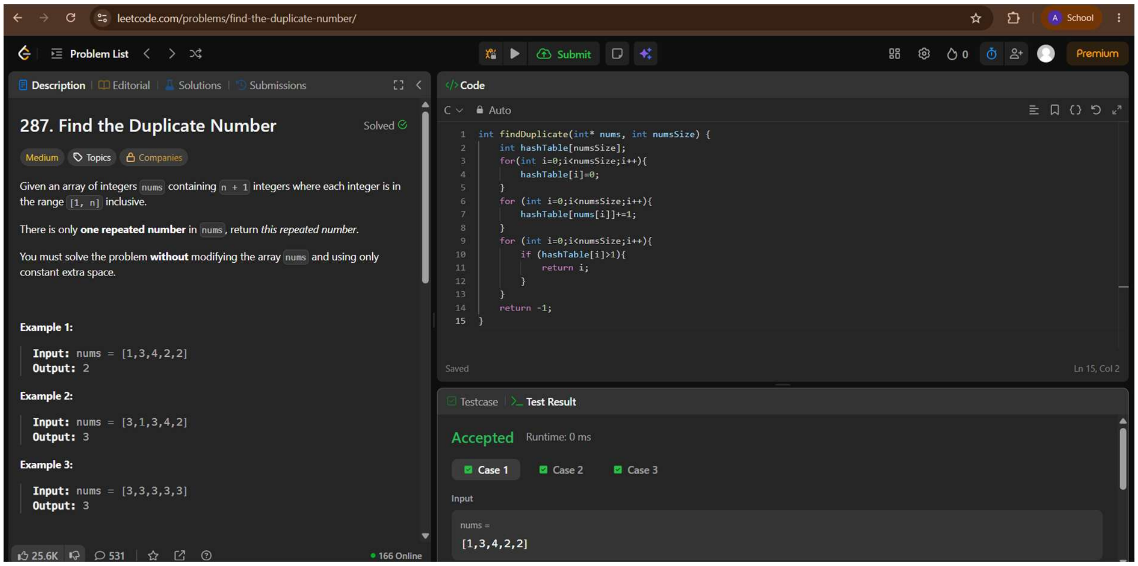
```
#include<stdio.h>
#include<math.h>

int main()
{
    int i, j, n, k;
    int cost[20][20];
    printf("Enter number of vertices:");
    scanf("%d", &n);
    printf("Enter cost matrix:");
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
            scanf("%d", &cost[i][j]);
    }
    for(k=0; k<n; k++)
    {
        for(i=0; i<n; i++)
        {
            for(j=0; j<n; j++)
            {
                if(cost[i][j] > cost[i][k]+cost[k][j])
                    cost[i][j] = cost[i][k]+cost[k][j];
            }
        }
    }
    printf("\nShortest path matrix:\n");
```

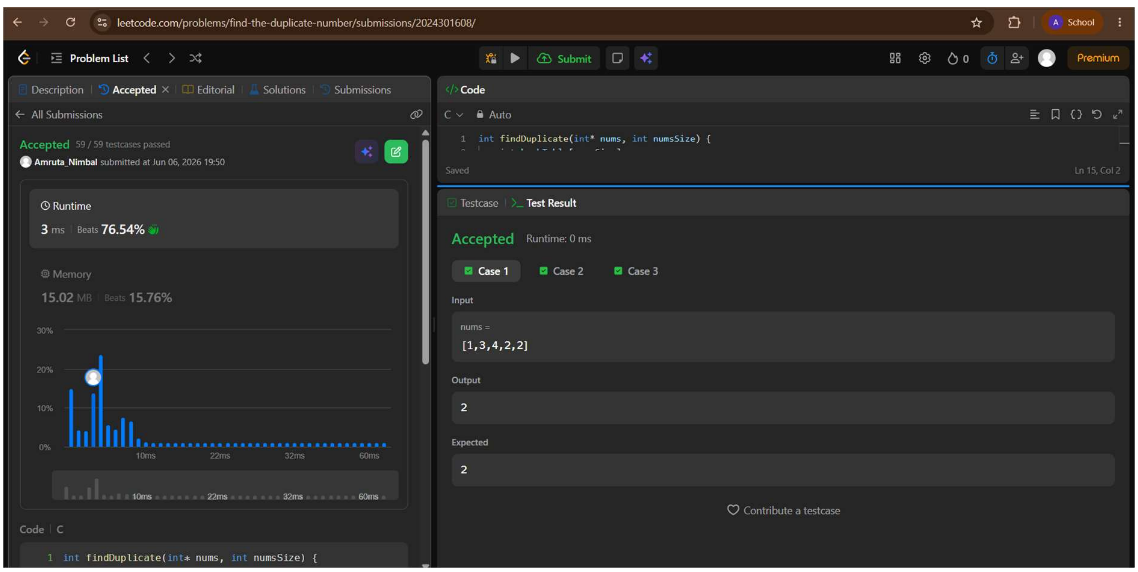
```
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
            printf("%d\t", cost[i][j]);
        printf("\n");
    }
    return 0;
}
```

Output:



Leetcode 287:





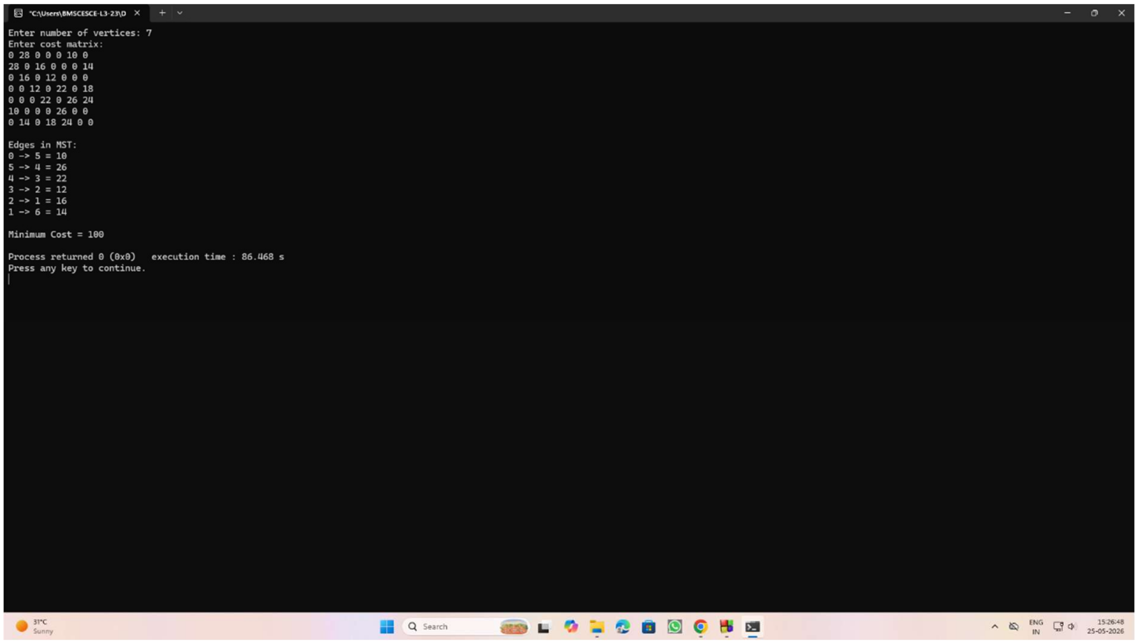
Lab Program 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
#define INF 999

int main()
{
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int cost[10][10];
    printf("Enter cost matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0 && i != j)
            {
                cost[i][j] = INF;
            }
        }
    }
    int visited[10] = {0};
    visited[0] = 1;
    int edges = 0;
    int minCost = 0;
    printf("\nEdges in MST:\n");
```

```
    while(edges < n - 1)
    {
        int min = INF;
        int u = -1, v = -1;
        for(int i = 0; i < n; i++)
        {
            if(visited[i])
            {
                for(int j = 0; j < n; j++)
                {
                    if(!visited[j] && cost[i][j] < min)
                    {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }
        printf("%d -> %d = %d\n", u, v, min);
        minCost += min;
        visited[v] = 1;
        edges++;
    }
    printf("\nMinimum Cost = %d\n", minCost);
    return 0;
}
```

Output:



```
Enter number of vertices: 7
Enter cost matrix:
0 28 0 0 0 10 0
28 0 16 0 0 0 14
0 16 0 12 0 0 0
0 0 12 0 22 0 18
0 0 0 22 0 26 24
10 0 0 0 26 0 0
0 14 0 18 24 0 0

Edges in MST:
0 -> 5 = 10
5 -> 4 = 26
4 -> 3 = 22
3 -> 2 = 12
2 -> 1 = 16
1 -> 6 = 14

Minimum Cost = 100

Process returned 0 (0x0)   execution time : 86.468 s
Press any key to continue.
```

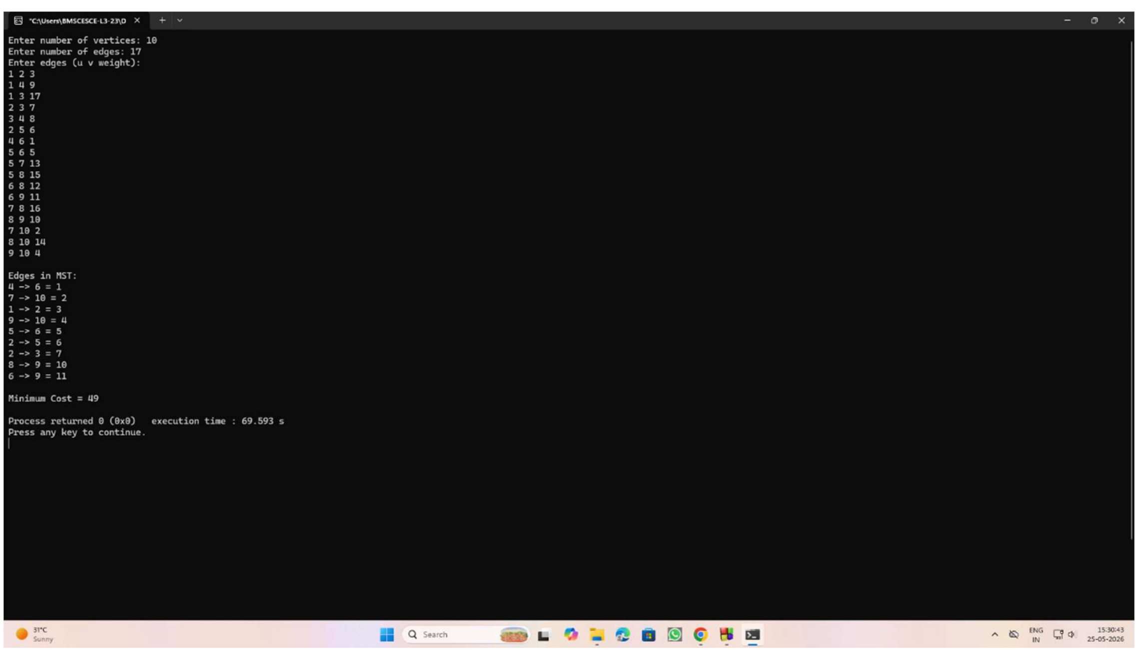
(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>


struct Edge
{
    int u, v, weight;
};


int parent[10];


int find(int i)
{
    while(parent[i] != i)
    {
        i = parent[i];
    }
    return i;
}


void unionSet(int a, int b)
{
    parent[a] = b;
}


int main()
{
    int n, e;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
```

```
printf("Enter number of edges: ");
scanf("%d", &e);
struct Edge edge[e];
printf("Enter edges (u v weight):\n");
for(int i = 0; i < e; i++)
{
    scanf("%d %d %d",
          &edge[i].u,
          &edge[i].v,
          &edge[i].weight);
}
for(int i = 0; i < e - 1; i++)
{
    for(int j = i + 1; j < e; j++)
    {
        if(edge[i].weight > edge[j].weight)
        {
            struct Edge temp = edge[i];
            edge[i] = edge[j];
            edge[j] = temp;
        }
    }
}
for(int i = 0; i < n; i++)
{
    parent[i] = i;
}
int minCost = 0;
printf("\nEdges in MST:\n");
```

```
    for(int i = 0; i < e; i++)
    {
        int a = find(edge[i].u);
        int b = find(edge[i].v);
        if(a != b)
        {
            printf("%d -> %d = %d\n",
                   edge[i].u,
                   edge[i].v,
                   edge[i].weight);
            minCost += edge[i].weight;
            unionSet(a, b);
        }
    }
    printf("\nMinimum Cost = %d\n", minCost);
    return 0;
}
```

Output:



Lab Program 9:

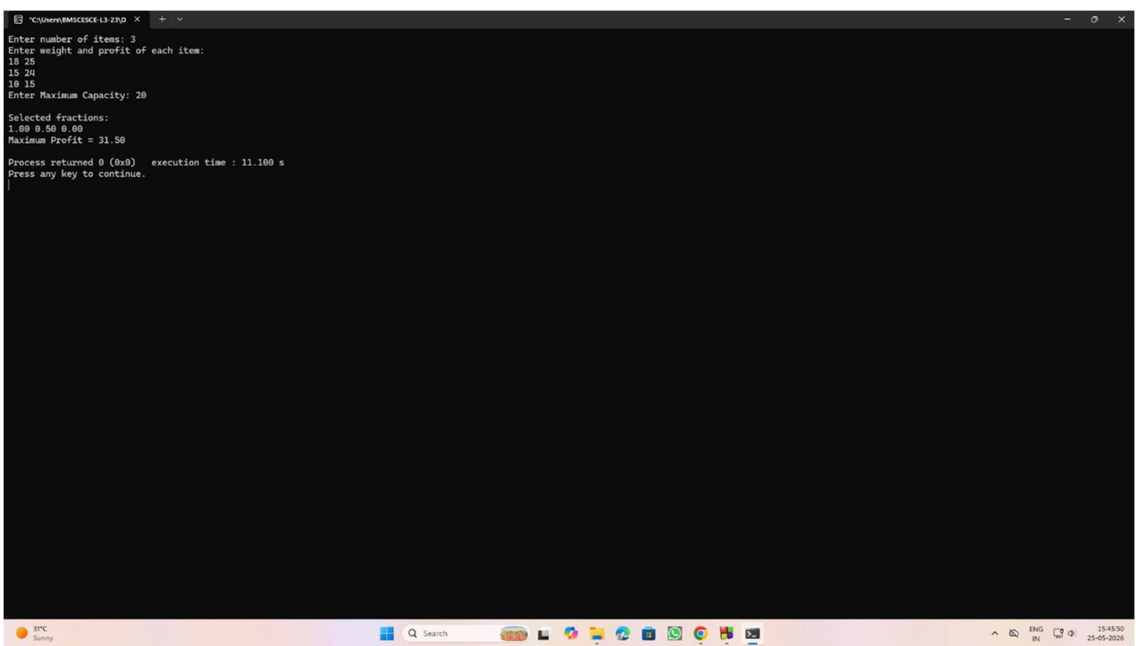
Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h>
float w[50], p[50], ratio[50], x[50];

void knapsack(int m, int n)
{
    int rem_cap;
    float totalProfit = 0.0;
    for(int i = 0; i < n; i++)
    {
        x[i] = 0.0;
    }
    rem_cap = m;
    for(int i = 0; i < n; i++)
    {
        if(w[i] <= rem_cap)
        {
            x[i] = 1.0;
            rem_cap = rem_cap - w[i];
            totalProfit = totalProfit + p[i];
        }
        else
        {
            x[i] = (float)rem_cap / w[i];
            totalProfit =
            totalProfit + (x[i] * p[i]);
            break;
        }
```

```
    }
    printf("\nSelected fractions:\n");
    for(int i = 0; i < n; i++)
    {
        printf("%.2f ", x[i]);
    }
    printf("\n");
    printf("Maximum Profit = %.2f\n", totalProfit);
}

int main()
{
    int n, m, i, j;
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter weight and profit of each item:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%f %f", &w[i], &p[i]);
        ratio[i] = p[i] / w[i];
    }
    printf("Enter Maximum Capacity: ");
    scanf("%d", &m);
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(ratio[i] < ratio[j])
            {
```

```
            float temp;
            temp = ratio[i];
            ratio[i] = ratio[j];
            ratio[j] = temp;
            temp = w[i];
            w[i] = w[j];
            w[j] = temp;
            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
         }
      }
   }
   knapsack(m, n);
   return 0;
}
```

Output:



Lab Program 10:

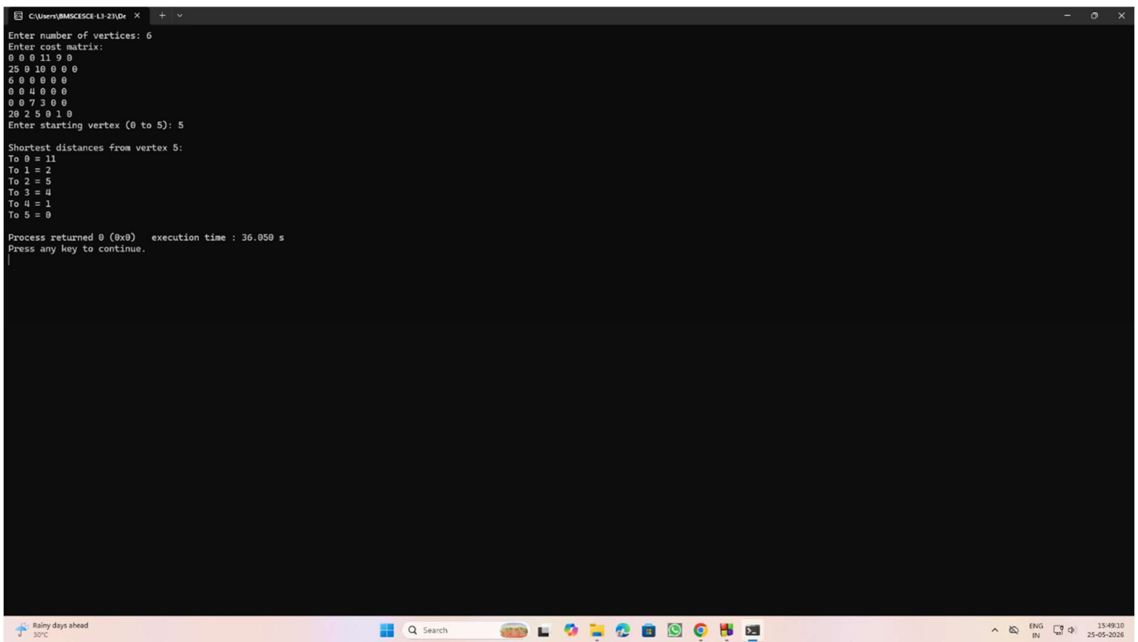
From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>
#define MAX 10
#define INF 9999

int main()
{
    int n, i, j, start;
    int cost[MAX][MAX];
    int dist[MAX];
    int visited[MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0 && i != j)
            {
                cost[i][j] = INF;
            }
        }
    }
    printf("Enter starting vertex (0 to %d): ", n - 1);
    scanf("%d", &start);
```

```
for(i = 0; i < n; i++)
{
    dist[i] = cost[start][i];

    visited[i] = 0;
}
dist[start] = 0;
visited[start] = 1;
for(i = 1; i < n; i++)
{
    int min = INF;
    int u = -1;
    for(j = 0; j < n; j++)
    {
        if(!visited[j] && dist[j] < min)
        {
            min = dist[j];
            u = j;
        }
    }
    if(u == -1)
        break;
    visited[u] = 1;
    for(j = 0; j < n; j++)
    {
        if(!visited[j] &&
           dist[u] + cost[u][j] < dist[j])
        {
            dist[j] =
            dist[u] + cost[u][j];
```

```
            }
        }
    }
    printf("\nShortest distances from vertex %d:\n", start);
    for(i = 0; i < n; i++)
    {
        printf("To %d = %d\n", i, dist[i]);
    }
    return 0;
}
```

Output:



Lab Program 11:

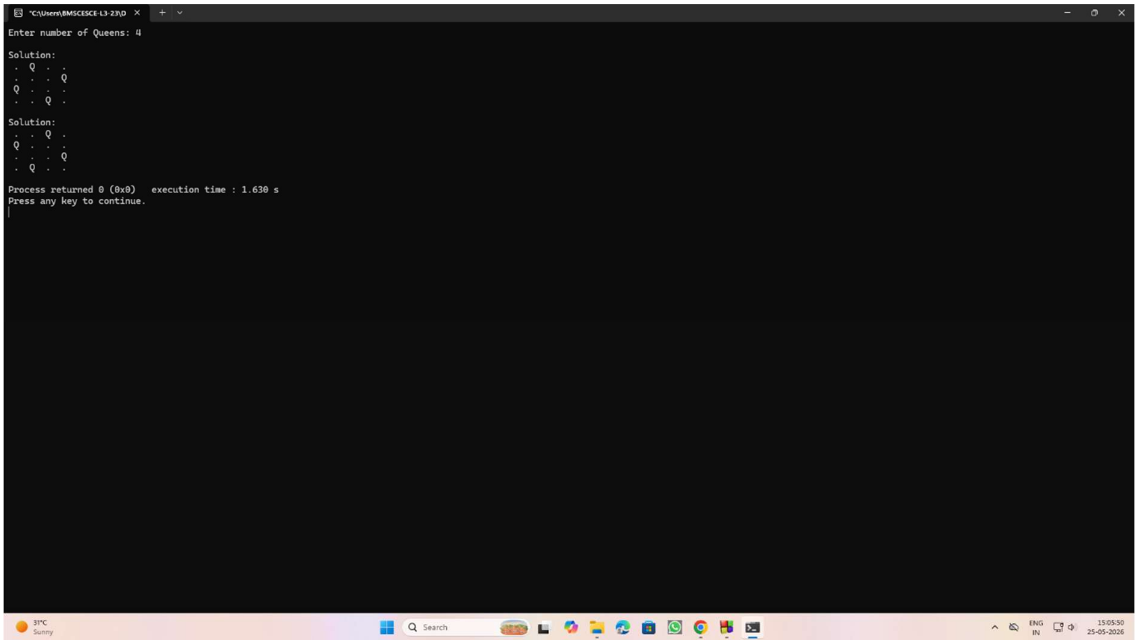
Implement "N-Queens Problem" using Backtracking.

```
#include<stdio.h>
#include<math.h>

int board[20], n;

int isSafe(int row, int col)
{
   for(int i=1; i<=row; i++)
   {
      if(board[i]==col || (abs(board[i]-col) == abs(i-row)))
         return 0;
   }
   return 1;
}

void printSolution()
{
   printf("\nSolution:\n");
   for(int i=1; i<=n; i++)
   {
      for(int j=1; j<=n; j++)
      {
         if(board[i] == j)
            printf("Q");
         else
```

```
            printf(".");
        }
        printf("\n");
    }
}

void solveNQueens(int row)
{
    for(int col=1; col<=n; col++)
    {
        if(isSafe(row, col))
        {
            board[row]=col;
            if(row==n)
                printSolution();
            else
                solveNQueens(row+1);
        }
    }
}

int main()
{
    printf("Enter number of Queens:");
    scanf("%d", &n);
    solveNQueens(1);
    return 0;
```

}

Output:


"C:\Users\BMSCESCE-L3-23\D
Enter number of Queens: 4
Solution:
. Q . .
. . . Q
Q . . .
. . Q .
Solution:
. . Q .
Q . . .
. . . Q
. Q . .
Process returned 0 (0x0)   execution time : 1.630 s
Press any key to continue.